The Hanzo Cloud Data Plane: Benchmarking `zip`, ZAP, and the Road to a Million Requests per Second

Zach Kelling

Hanzo AI

2026

Abstract

We present a reproducible benchmark of the Hanzo cloud data plane — the `zip` web framework and its native ZAP binary transport — and a systems-tuning study on a two-node 2.5 GbE testbed (an NVIDIA GB10 "spark" server, 20 cores, aarch64; a 32-core x86_64 "evo" load generator). We establish that `zip` carries a negligible tax over the `fasthttp` engine it wraps (≈24 ns and one 48-byte allocation per request; 0.95× raw throughput on a no-op handler). We then show that the naive distributed result — 194,000 req/s over the wire — is not a CPU or bandwidth limit but a *single hardware RX-queue* on the 2.5 GbE NIC, whose driver refuses hardware multi-queue. Enabling Receive Packet Steering (RPS) plus a deeper backlog and larger socket buffers lifts the same link to **518,000 req/s (a 2.67× gain)**, at which point the tuned network result exceeds loopback because the server's cores no longer contend with the load generator. Finally we characterize the ZAP transport. It initially ran *slower* than HTTP (91k vs. 481k req/s loopback), which we localize by construction to a non-zero-allocation codec — three allocation sites, the largest an unpooled request context, driving 2203 garbage collections in six seconds (relaxing `GOGC` 100 → 800 alone more than doubles throughput). Pooling the context and making the marshalers zero-allocation — a pure codec rewrite with a byte-identical wire — lifts ZAP 6.3× to **570k req/s, overtaking HTTP by** 1.18×, with GC falling 2203 → 62. We show this is a CPU/allocation win, not a wire win on trivial payloads (the ZAP frame is *larger* there: 124 vs. 75 bytes), and that over the saturated 2.5 GbE link both transports tie at ≈530k. Separately, on the serialization axis — native typed RPC vs. HTTP+JSON on a shared record — ZAP *decodes* 35–174× faster at zero allocations (13 ns vs. 473–2334 ns), a wash on encode, netting 1.1–1.3× end-to-end once the socket dominates. We release the framework as `hanzoai/benchmarks` and the tuning as a single idempotent script.

1 Introduction

The Hanzo control plane ships as a single Go binary that mounts every subsystem (AI gateway, commerce, IAM, KMS) and serves `api.hanzo.ai` [1]. Its HTTP surface is `zip` [2], a framework built on a fork of `fiber` / `fasthttp`, and its inter-subsystem calls travel over *ZAP*, a typed binary transport that reuses `fasthttp` request and response objects over a length-framed wire [3]. Two questions follow naturally: *(i)* how much does the framework cost over the bare engine, and *(ii)* where is the real ceiling of the data plane on modern hardware — is it the framework, the CPU, the network stack, or the link?

Benchmarking answers only when it is fair. We hold the handler, the payload, the concurrency, and the tuning identical across every measurement; we report allocations, not just throughput; and we separate the *load generator's* limits from the *server's*. All artifacts, scripts, and raw numbers are public in `hanzoai/benchmarks`.

2 Methodology

Testbed. *spark* is an NVIDIA GB10 (Grace–Blackwell) developer system: 20 aarch64 cores, running the server. *evo* is a 32-core x86_64 box, running the load generator. They are joined by a direct 2.5 GbE wired link (`enP7s7`↔`eno1`, both `Speed: 2500Mb/s`, subnet 192.168.77.0/24) — confirmed with `ethtool`, distinct from the WiFi path used for control SSH. A second host, *dbc*, is wired on a 1 GbE USB-ethernet port for future dual-loader aggregation.

Framework. `hanzoai/benchmarks` is two-tier. The *property/micro* tier (`cloud/shard`, `framework/zip`) is pure Go, deterministic, and cluster-free, so it runs in CI and isolates a primitive (routing ns/op) or proves an invariant. The *integration* tier (`k3s/`, `distributed.sh`) stands the real binary up and drives it over the network. Load is generated with `bombardier` (a `fasthttp`-based client that actually saturates the server; `hey`, a `net/http` client, caps near

Table 1: End-to-end HTTP, loopback, spark, `-c 125`. Same handler.

framework	req/s	vs. fasthttp
raw `fasthttp`	365–400k	1.00×
`zip` (HTTP)	338–348k	0.95×

100k and is unsuitable) and, for ZAP, with a purpose-built client (§6) since no HTTP tool speaks the protocol.

Fairness. Every server exposes the identical handler — `GET /health` returning the two-byte body `"ok"`. Connections are warm (keep-alive) so we measure steady-state, not handshakes. We report the median of sustained throughput and the p50/p99 latency; where garbage collection is implicated we report allocations per operation from `-benchmem`.

3 The Framework Tax: `zip` vs. `fasthttp`

The in-process cost of `zip` over raw `fiber` is a constant ≈24 ns and exactly one 48-byte allocation (the `*zip.Ctx`), independent of route shape [4]; on a handler doing ∼3500 ns of real work it is under 1%. End to end, driven by `bombardier` on loopback (Table 1), `zip` sustains 0.95× of raw `fasthttp` — the wrapper is noise.

4 The Network Ceiling Is Not the Box

Moving the load off-box *lowered* throughput — and adding connections made it worse (Table 2). Throughput fell from 194k at `-c 1000` to 154k at `-c 8000` while p99 latency rose 5 → 52 ms: classic congestion collapse. Yet the link ran at ∼70 MB/s, only 22% of the 2.5 GbE bandwidth, and spark's cores sat mostly idle. By Little's law the 194k/5.1 ms operating point is latency-bound, not bandwidth-bound.

The root cause is structural: `ethtool -l` reports the NIC configured with a *single hardware RX queue*, and the driver rejects reconfiguration (`ethtool -L`: "Operation not supported"). All inbound packets are therefore serviced by one core's softirq, and that single core is the ceiling.

Table 2: Untuned evo→spark over 2.5 GbE. Congestion collapse.

loader → target	req/s	p99 latency
loopback (`-c 500`)	365k	—
evo→spark `-c 1000`	194k	5.1 ms
evo→spark `-c 4000`	167k	24 ms
evo→spark `-c 8000`	154k	52 ms

Table 3: evo→spark, both ends tuned. Same link, same handler.

concurrency	untuned	tuned	gain
`-c 1000`	194k	518k	2.67×
`-c 2000`	collapsed	518k	—
`-c 4000`	154k	474k (peak 696k)	3.1×

5 System Tuning: 2.67× on the Same Wire

The software remedy for a single-queue NIC is Receive Packet Steering, which hashes flows across cores in the kernel. We apply RPS (and RFS/XPS) across all 20 cores, raise `net.core.netdev_max_backlog` from 1000 to 300000, set 256 MiB socket buffers, widen the ephemeral port range, and pin the performance governor — all in one idempotent `tune.sh`, run on server and loader alike. The result (Table 3) is 2.67×: the same 2.5 GbE link now sustains **518k req/s**, peaking at 696k.

Two observations. First, the tuned *network* figure (518k) now exceeds the loopback figure (365–400k): over the wire spark's 20 cores serve full-time, whereas on loopback they are shared with the load generator. At 518k spark idles only ∼12%, so this is close to the true single-loader serving ceiling; the remaining headroom is the province of a second, parallel NIC path.

6 The ZAP Transport: A Zero-Alloc Codec Overtakes HTTP

`zip` is ZAP-native — a bare listen address binds the binary transport; `http://` binds `fasthttp`. Because HTTP tools cannot speak ZAP we built `zapbench`, a concurrent client over `zaphttp.Transport`. The first result inverted the expectation: ZAP delivered only 91k req/s on loopback against HTTP's 481k — ZAP *lost* by 5×.

The cause was not the wire but the codec. The tail (p50 303 µs but p99 13 ms) is a garbage-collection signature, and relaxing `GOGC` 100 → 800 more than doubled ZAP to 203k — proof the path was allocation-bound. Three allocation sites carried it. `MarshalRequest`/`MarshalResponse` minted a fresh frame slice per call; header decode performed `string()` copies into a `map[string][]string`; and structurally, `zaphttp`'s server allocated a fresh `fasthttp.RequestCtx` *per request* (`server.go`), where `fasthttp`'s own server pools it per connection. Over a six-second run the ZAP server triggered **2203** garbage collections; the HTTP server, 57.

The fix (`zap-proto/http`, branch `perf/zero-alloc-codec`) is mechanical, not architectural: pool the `RequestCtx` once per connection, hand-roll a frame encoder that appends into a pooled buffer, and drain the 4-byte length prefix through `bufio.Peek`/`Discard` so no header array escapes to the heap. The serve path serializes with `AppendResponse` into that reused buffer — **zero** allocations, versus 19 for the old fresh-slice `MarshalResponse` — and `UnmarshalRequest` decodes headers in place, also zero-alloc; over a warm connection the serve loop allocates nothing per request. Golden tests confirm the wire is *byte-identical* before and after — this is a pure codec rewrite, no protocol change. The measured effect (Table 4): ZAP rises 91k → 570k req/s, a **6.3**× lift that **overtakes HTTP at 481k — ZAP wins** 1.18× on loopback, with p99 collapsing 13 ms → 1.19 ms and GC falling 2203 → 62.

Table 4: ZAP vs. HTTP, loopback, `-c 125`, same handler. The zero-alloc codec flips ZAP from a 5× loss to a 1.18× win.

transport	req/s	note
`zip` HTTP (fasthttp)	481k	zero-alloc; 57 GC
`zip` ZAP (`GOGC=100`)	91k	alloc-bound; 2203 GC
`zip` ZAP (`GOGC=800`)	203k	2.2×; confirms cause
`zip` **ZAP zero-alloc**	**570k**	**0 allocs/req; 62 GC;** 1.18×

Two corrections to intuition survive the fix, and both matter for where ZAP is actually worth deploying. *First, the win is CPU, not the wire.* Measured by `TestWireBytes`, the ZAP frame is *larger* than HTTP (124 vs. 75 bytes for a `/health` request); ZAP overtakes HTTP by spending fewer cycles per byte, not fewer bytes. *Second, `zaphttp` does not skip JSON.* It carries the HTTP body as opaque bytes on both transports, so a handler pays the same (de)serialization cost either way; on a chat-shaped JSON payload ZAP is in fact 0.95× HTTP, a slight loss from copying the body into the frame tail. JSON-elision is a property of *native* ZAP typed RPC — a separate axis `zaphttp` does not exercise. The loopback win is therefore real and purely a CPU/allocation win; over the 2.5 GbE link both transports tie at ≈530k, saturating the wire (§4) where the codec is no longer the bottleneck.

7 Skipping JSON: Typed RPC vs. HTTP+JSON

Section 6 isolates *framing* CPU: `zaphttp` carries the HTTP body opaque, so both transports pay the same JSON cost and ZAP's win is the zero-alloc frame codec alone. That leaves ZAP's larger structural claim untested — that a *native* typed binary message is read field-by-field off the wire with no parse step, where JSON must scan text and reflect. We test it directly: one shared record (an `eth_getBalance`-shaped call, four fields each way) round-tripped by `encoding/json` (stdlib), by `goccy/go-json`

and `sonic` (the fastest third-party JSON), and by `zap-proto/go`'s typed builder/reader. Same fields in, same compute, same fields out; a round-trip-equivalence test enforces byte-exact re-encoding and that every field is consumed on both sides, so neither codec skips work the other does.

The decode path is the headline (Table 5). Reading the four-field request costs **13 ns and zero allocations** in ZAP — a zero-copy read at a fixed offset — against 473 ns/4 allocs for the fastest JSON (`goccy`) and 2334 ns/13 allocs for stdlib: a **35× to 174×** gap, and the difference between allocating nothing and allocating per field. This is the "skip JSON" advantage made concrete — there is no parse, only a pointer and an offset.

Two honest counterweights. *Encode is a wash*, not a win: ZAP serializes in 629 ns with *more* allocations than JSON (12 vs. 4), because the published `v1.3.0` builder defers variable-length tails through a copy and escapes its object builder to the heap — the pooled fixed-width API that removes this is not in the released tag, so we report the tag-reproducible number. Netted out, the server handler (decode + compute + encode) still favors ZAP at 274 ns vs. `goccy`'s 431 ns, because the decode margin dominates the encode wash. *And end-to-end the margin compresses*: over the loopback `/rpc` endpoint ZAP serves 535k req/s against `sonic`'s 483k (1.11×), `goccy`'s 452k (1.18×), and stdlib's 407k (1.31×) — real, but far below the 174× microbenchmark gap, because at half a million requests per second the socket and scheduler dominate and serialization is a shrinking slice of per-request cost. Finally, on a structured record ZAP's wire is also *smaller* (162 vs. 233 bytes total): JSON's hex-string addresses and decimal integers are verbose. This refines §6 — ZAP's frame is larger only on trivial (`/health`-sized) payloads, and smaller once the record carries real typed fields.

Table 5: Serialization only, one shared 4-field record, GB10, median of 3, default `GOGC`. Decode is where typed-binary skips JSON entirely; the round-trip is fastest on ZAP though it allocates more than `goccy`.

codec	decode (ns)	round-trip (ns)	round-trip alloc
JSON stdlib	2334	3063	1
JSON `sonic`	845	1460	1
JSON `goccy`	473	904	
ZAP typed	**13.4**	**630**	1

8 Aside: Horizontal-Scale Routing

The same data plane shards per-tenant SQLite state across pods by highest-random-weight (rendezvous) election over a static membership set. A million synthetic tenants distribute across pods with 0.05% standard deviation at N=3 (no hot shard), every tenant resolves to exactly one deterministic owner (the no-two-writers invariant, verified over 10^5 tenants × 6 re-elections), a $4 \to 5$ scale-up remaps only 20.1% of tenants (the rendezvous ideal), and the routing itself costs ≈545 ns per request. Horizontal writer scale is thus gated on I/O, not routing.

9 Conclusions and Future Work

The Hanzo data plane is not framework-bound (`zip` ≈ `fasthttp`), and on a single 2.5 GbE link it serves 518k req/s once the kernel's receive path is tuned — a 2.67× gain from software steering alone, with the box only ∼12% idle. The ZAP codec is now zero-allocation and overtakes HTTP by 1.18× on loopback (§6), and its native typed RPC decodes structured records 35–174× faster than JSON at zero allocations (§7) — a decode advantage that today is masked end-to-end by socket overhead and a not-yet-zero-alloc encode path, and that a pooled fixed-width builder would surface. The remaining levers to seven figures are orthogonal and physical: a *dual-loader* across the two physi-

cal NICs (parallel RX paths), faster interconnect (10 GbE/RDMA), and a `writev` split so ZAP's frame tail avoids one body copy on large payloads. We also intend to benchmark the post-quantum transports natively available in Go 1.26 (`X25519MLKEM768`): PQ-TLS 1.3, ZAP-over-PQ-TLS, and PQ-QUIC, separating handshake cost (ML-KEM) from steady-state AEAD throughput.

References

[1] Hanzo Industries. *HIP-0106: The Unified Cloud Binary*. 2026.

[2] `zap-proto/zip` — the ZAP-native web framework. `https://github.com/zap-proto/zip`.

[3] `zap-proto/http` — ZAP-HTTP binary transport. `https://github.com/zap-proto/http`.

[4] `zap-proto/zip BENCHMARKS.md` — framework wrapper tax.